

内存溢出可以使用 SoftReference，内存泄露可以使用 WeakReference，GC 监控可以使用 PhantomReference

内存溢出

内存溢出 OOM Out Of Memory：内存溢出并不是说内存不足，而是说应用程序的 **申请内存**，超出类 DVM 所允许的 **最大可使用内存** (Dalvik VM HeapGrowthLimit)

- 配置清单文件的 **largeHeap** 属性，可以分配到更多的内存

内存溢出的原因

1. **大内存对象**

例如 Bitmap，如果图片过大，容易造成 OOM：我们可以对 Bitmap 进行压缩、或者使用软引用进行包装，并且记得使用完毕后及时回收

例如 Thread，如果开启的子线程太多，容易造成 OOM：我们可以使用线程池进行统一管理

2. **内存泄露**

内存泄露过多，也会造成内存溢出

3. **优化数据结构**

SparseArray/ArrayMap 替代 HashMap 可以节省部分内存

StringBuilder 代替 String 避免对象的频繁创建

int 代替 Enum 节约部分内存

4. 短时间内创建了大量对象（这个往往是由于代码不规范造成的）

递归次数过多，或者无限循环（短时间）

ListView 的优化：复用 + 图片优化（短时间）

内存溢出的方式

1. 栈溢出
2. 堆溢出
3. 常量池溢出

内存溢出代码

栈溢出

方法的无限递归调用：StackOverflow

开启的线程太多占满栈内存：OutOfMemoryError

```
// StackOverflow
public void stackLeak(){
    stackLeak();
}

public static void main(String[] args) {
    try{
```

```

        stackLeak();
    }catch(Exception e){
        System.out.println("Stack length:");
    }
}

// OutOfMemoryError
public static void main(String[] args) {
    try{
        while(true){
            new Thread(){
                @Override
                public void run() {
                    dontStop(); // 又是一个while循环，线程停不下来
                }
            }.start()
        }
    }catch(Exception e){
        System.out.println("Stack length:");
    }
}

```

堆溢出

短期内，创建大量的对象

```

List<Object> list = new ArrayList<>();
while(true){
    list.add(new Object());
}

```

常量池溢出

通过 `String.intern()` 方法，将字符串的值放到常量池中

报错： `OutOfMemoryError:PermGen Space` （永久代）

```
int i = 0;
List<String> list = new ArrayList<>();
while(true){
    list.add(String.valueOf(i++).intern());
}
```

Bitmap

Bitmap 非常 **消耗内存资源**，而且在读取 Bitmap 的时候，虚拟机分配的 **图片堆栈只有 8M**，所以经常造成 OOM 异常

Bitmap 的优化：

图片显示（缩略图）

图片显示：加载合适尺寸的图片，比如显示 **缩略图** 的地方不要加载大图

图片大小：长度×宽度×单位像素所占据字节数

图片压缩

图片压缩：直接使用 ImageView 显示 Bitmap 时会占很多资源，尤其当图片较大时容易发生OOM，可以使用 `BitmapFactory.Options` 对图片进行压缩。

图片回收

图片回收：使用完 Bitmap，及时使用 `Bitmap.recycle()` 回收

问题：Android 不是自身具备垃圾回收机制吗？此处为何要手动回收？

- Bitmap 对象不是 new 生成的，而是通过 `BitmapFactory` 生产的
- 通过源码可发现 `BitmapFactory` 是通过 **调用 JNI 生成 Bitmap 对象**（`nativeDecodeStream` 等方法）
- 所以加载 Bitmap 到内存包括两部分，**Dalvik 内存** 和 **Linux kernel 内存**，前者会被虚拟机自动回收，而后者必须通过 `recycle()` 方法进行回收（内部调用 `nativeRecycle`）

图片像素

图片像素：Android 默认颜色模式为 **ARGB_8888**，显示质量最高、占用内存最大， 若要求不高时可采用 **RGB_565** 等模式

模式	描述	占用字节
ARGB_8888 (默认)	4 个 8 为组成，共 32 位，每个色彩元素占 8 位	4B
ARGB_4444	4 个 4 位组成，共 16 位，每个色彩元素占 4 位	2B
RGB_565	R 为 5 位，G 为 6 位，B 为 5 位，共 16 位，没有 Alpha	2B
ALPHA	Alpha 由 8 位组成	1B

其他

比如采用 **软引用**、**图片缓存**、**对 OOM 异常进行捕获** 等等

缓冲池

对象池：如果某个对象的创建，非常消耗内存资源，可以建立一个对象池缓存已创建的对象

线程池：线程的创建，非常消耗内存资源，可以建立一个线程池统一管理所有的线程

内存泄漏

内存泄漏：本该被回收的对象，由于另外一个正在使用的对象持有它的引用，而导致它不能被正常回收

- 内存泄漏本身不会造成 OOM 异常，但是内存泄漏过多，达到临界值的时候会诱发 OOM

内存泄露的原因

1. **static 造成的内存泄露（单例）**

- static 修饰的方法和变量不被 GC 管理
- static 修饰 Context 变量，这个时候应该尽量使用 Application 的上下文，因为它生命周期比较长，不会造成内存泄漏
- 使用软引用

2. **非静态内部类**

- 非静态内部类会持有外部类的一个引用，如果内部类没有执行完毕，外部类也得不到释放

3. **Handler** 造成的内存泄漏
4. **线程 Thread** 造成的内存泄漏
5. **应当关闭的资源**，没有及时关闭
6. **应当注销的对象**，没有及时注销

单例造成的内存泄漏

单例的静态特性，使的它的生命周期和应用的生命周期一样长，如果一个对象已经不需要使用了，而单例还持有该对象的引用，就会导致该对象不能被正常回收，造成内存泄漏

单例造成 Context 的内存泄漏

利用 **弱引用**，可以避免内存泄漏

利用 **Application 的上下文**，也可以避免内存泄漏

```
public class AppManager {  
    private static AppManager instance;  
    private Context context;    // 利用弱引用，可以避免内存泄漏  
    private AppManager(Context context) {  
        this.context = context;  
        this.context = context.getApplicationContext();    //  
    }  
    public static AppManager getInstance(Context context) {
```



```
if (instance != null) {  
    instance = new AppManager(context);  
}  
return instance;  
}  
}
```

非静态内部类造成的内存泄漏

非静态内部类会持有外部类的一个引用，如果一个外部类不需要使用了，而内部类的某个对象依然存在使用，那么就会造成外部类的内存泄漏

- 定义成 静态内部类、或者 单独定义一个类，可以避免内存泄漏

非静态内部类造成 Activity 的内存泄漏

```
public class MainActivity extends AppCompatActivity {  
    private static TestResource mResource = null;    // 该静态实例会一直持有 Activity 的引用  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        if(mResource == null){  
            mResource = new TestResource();  
        }  
    }  
}
```

```
    }  
}  
  
// 非静态内部类  
class TestResource {  
  
}  
}
```

Handler 造成的内存泄漏

Handler 造成的内存泄漏应该说最为常见了，它造成内存泄漏原因主要有 2 点：

1. Handler 会持有外部类的一个引用
 - 我们经常将 Handler 定义成一个非静态匿名内部类（非静态内部类会持有外部类的一个引用）
2. Message 会持有 Handler 的一个引用
 - 消息队列是在一个 Looper 线程中不断轮询处理的，它是一个 **全局的静态单例对象**
 - 只要还有消息未处理或者正在处理（Message 对象 **依然在消息队列** 中），那么该 Message 就一直得不到释放，造成 Handler 的内存泄漏

解决 Handler 造成的内存泄漏：

1. 将 Handler 定义成一个 **静态内部类/或者外部类** 的形式
2. 在 Activity 销毁的时候, 及时调用 `removeCallbacksAndMessages(null)` 移除未处理的消息
3. 如果需要使用 Context 上下文对象 (Handler 所使用的对象), 将其定义成一个 **弱引用 WeakReference**

Handler 造成 Activity 的内存泄漏

```
public class MainActivity extends AppCompatActivity {  
    // 非静态匿名内部类, 会持有外部类 Activity 的一个引用  
    private Handler mHandler = new Handler() {  
        @Override  
        public void handleMessage(Message msg) {  
            //...  
        }  
    };  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        loadData();  
    }  
  
    private void loadData(){  
        Message message = Message.obtain();  
        mHandler.sendMessage(message);  
    }  
}
```

```
}
```

```
}
```

解决 Handler 造成的内存泄漏

```
public class MainActivity extends AppCompatActivity {
    private MyHandler mHandler = new MyHandler(this);
    private TextView mTextView ;

    // 将 Handler 定义成一个静态内部类
    private static class MyHandler extends Handler {
        private WeakReference<Context> reference;    // 弱引用
        public MyHandler(Context context) {
            reference = new WeakReference<>(context);
        }
        @Override
        public void handleMessage(Message msg) {
            MainActivity activity = (MainActivity)
reference.get();
            if(activity != null){
                activity.mTextView.setText("");
            }
        }
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mTextView = (TextView)findViewById(R.id.textview);
    }
}
```

```
loadData();
```

```
}
```

```
private void loadData() {
```

```
    Message message = Message.obtain();
```

```
    mHandler.sendMessage(message);
```

```
}
```

```
@Override
```

```
protected void onDestroy() {
```

```
    super.onDestroy();
```

```
    mHandler.removeCallbacksAndMessages(null);    // 及时移
```

```
除消息
```

```
}
```

```
}
```

线程造成的内存泄漏

在使用线程的时候，**异步任务 AsyncTask** 和 **Runnable 对象**，我们通常都会定义成一个非静态匿名类；如果在 Activity 销毁前，该线程任务还未完成，就会造成内存泄漏

- 采用静态内部类或者外部类的形式
- 在 Activity 销毁的时候，取消异步任务

```
AsyncTask.cancel()
```

线程造成的内存泄漏

```

static class MyAsyncTask extends AsyncTask<Void, Void, Void>
{
    private WeakReference<Context> weakReference;

    public MyAsyncTask(Context context) {
        weakReference = new WeakReference<>(context);
    }

    @Override
    protected Void doInBackground(Void... params) {
        SystemClock.sleep(10000);
        return null;
    }

    @Override
    protected void onPostExecute(Void aVoid) {
        super.onPostExecute(aVoid);
        MainActivity activity = (MainActivity)
weakReference.get();
        if (activity != null) {
            //...
        }
    }
}

static class MyRunnable implements Runnable{
    @Override
    public void run() {
        SystemClock.sleep(10000);
    }
}

//—————
new Thread(new MyRunnable()).start();

```

```
new MyAsyncTask(this).execute();  
调用 AsyncTask.cancel() 取消任务
```

// 在 Activity 销毁的时候,

其他

应当关闭的资源未及时关闭

- 数据库：数据库的 **Cursor**、**SQLiteDatabase** 没有及时关闭（如果查询数据库得到的数据量比较小，是不会造成内存溢出的；但是如果太大的话就容易发生这种异常）
- IO 流：**IO 流** 没有及时关闭（内存和硬盘一直存在着连接占用着资源）

应当注销的对象未及时注销

- 广播 **BroadcastReceiver**：动态注册广播 `registerReceiver` 后，没有及时注销广播 `unRegisterReceiver`
- **EventBus**：注册 `register`，注销 `unRegister`
- 观察者 **Observer**：`ContentObserver`、`FileObserver` 等等

不要对 Activity、Service 等有自己生命周期的对象，直接引用作为成员变量，如果不得不这么做，尽量用 **弱引用** **WeakReference** 进行包裹

内存抖动

内存抖动：短时间内发生了多次内存分配和释放

- 主要原因是：短时间内 **频繁的创建对象**
 - 为了应对这种情况，虚拟机会触发 **频繁的 GC**
 - 当 GC 发生时，**其他线程会被挂起**，导致界面卡顿
-

ANR

ANR (Application Not Response)：**主线程** 长时间被阻塞，而导致应用程序无响应

- 当前事件没有机会得到处理 / 当前线程正在处理，但没有及时完成

ANR 主要有 3 钟类型：

- **KeyDispatchTimeout** (5s)：按键或触摸事件在特定时间内无响应
- **BroadcastTimeout** (10s)：BroadcastReceiver 在特定时间内无法完成处理（有序广播）
- **ServiceTimeout** (20s)：Service 在特定时间内无法完成

处理（小概率）

ANR 执行流程

了解 ANR 的执行流程，有助于我们制定 ANR 监控策略和获取 ANR 相关信息

- 系统捕获到 ANR
- Process 依次向本进程和其他进程发送 Linux 信号量 3
- 写入日志：进程收到 Linux 信号量，并向 `data/anr/traces.txt` 写入进程信息
- 打印日志：Log 日志 打印 ANR 信息
- 进程进入 ANR 状态（此时可以获取到进程的 ANR 信息）
- 弹出 ANR 提示框
- 提示框消失，进程恢复正常

由于向 `data/anr/traces.txt` 写入信息耗时较长，从 ANR 发生到弹出 ANR 提示框一般在 10s 左右（不同 Rom 时间不同）

- 导出 traces.txt 文件：`$ adb pull data/anr/traces.txt .`

如何定位 ANR 信息

- Log 日志
- 分析 traces.txt 文件

ANR 发生流程

输入时间导致的 ANR 流程

- 系统输入管理服务进程 `InputManagerService` 中有一个线程 `InputDispatcherThread` 专门管理输入事件的分发
- 在这个线程中 `InputDispatcher` 对象会不断的检测处理过程是否超时
- 一旦超时，就会回调 `InputMethod` 对象的 `notifyANR` 方法弹出 ANR 对话框（AMS 通过 handler 发送）

广播导致的 ANR 流程

- 广播分为普通的、有序的、异步的，其中只有 `有序广播` 才会发生 ANR 异常
- 在清单文件中注册的广播默认都是有序广播，它们会被放在一个 `广播队列` 中 `BroadcastQueue` 串行处理
- AMS 在处理广播的时候，会设置一个超时时间，一旦超时 `BroadcastQueue` 就会触发 `processNextQueue` 方法判断是否超时，如果超时终止广播并弹出 ANR 对话框